

**MÁSTER UNIVERSITARIO EN INGENIERÍA DE
TELECOMUNICACIÓN**

TRABAJO FIN DE MÁSTER

**AN AUTOMATED FRAMEWORK FOR DATASET QUALITY
ASSESSMENT
AND DATA LEAKAGE DETECTION IN MACHINE LEARNING**

JAIME CANO MORAÑO

2025–26

**UNIVERSIDAD POLITÉCNICA DE MADRID
ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN**

**MÁSTER UNIVERSITARIO EN INGENIERÍA DE
TELECOMUNICACIÓN**

TRABAJO FIN DE MÁSTER

**AN AUTOMATED FRAMEWORK FOR DATASET QUALITY ASSESSMENT
AND DATA LEAKAGE DETECTION IN MACHINE LEARNING**

JAIME CANO MORAÑO

2025–26

TRABAJO FIN DE MÁSTER

Título: An automated framework for dataset quality assessment and data leakage detection in machine learning
Autor: D. Jaime Cano Morano
Tutor: D. Yong Zheng
Ponente: D. Pedro Reviriego Vasallo
Departamento: Departamento de Ingeniería de Sistemas Telemáticos

Miembros del Tribunal

Presidente: D.
Vocal: D.
Secretario: D.
Suplente: D.

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de:

Madrid, a de de 20...

RESUMEN

El aprendizaje automático se ha convertido en una tecnología central en numerosos dominios de alto impacto. Sin embargo, la fiabilidad de los modelos entrenados depende de manera crítica de la calidad de los datos utilizados. Dos problemas especialmente dañinos son los defectos de calidad de los datos — como valores faltantes, duplicados, desequilibrio de clases o características de baja varianza — y la fuga de datos (*data leakage*), que ocurre cuando información que no estaría disponible en el momento de la predicción se filtra de forma inadvertida en el proceso de entrenamiento.

En este trabajo se presenta **ml-framework**, un sistema de código abierto para la evaluación automática de la calidad de conjuntos de datos y la detección de fuga de datos en flujos de trabajo de aprendizaje automático. El framework implementa un pipeline de 22 comprobaciones que cubre seis dimensiones: calidad, fuga, análisis de características, suficiencia estadística, detección de deriva e impacto.

La contribución más novedosa es la *puntuación unificada de riesgo de fuga*, $\mathcal{L}(f) = 0.35 \rho(f) + 0.35 \tilde{I}(f; y) + 0.30 \pi(f)$, que combina correlación de Pearson/Cramér, información mutua normalizada e inflación del rendimiento en una sola puntuación interpretable. Complementariamente, un módulo basado en GPT-4o-mini (Azure OpenAI) detecta fuga semántica — casos en los que el nombre de una característica revela información futura o *post hoc* que los métodos estadísticos no pueden capturar.

La evaluación sobre once conjuntos de datos (seis sintéticos y cinco de UCI) demuestra una tasa de detección de 4/4 en escenarios de fuga contruidos, puntuaciones de disponibilidad calibradas de 65 (C, datos sucios) a 95 (A, control limpio), y cobertura de $3.2\times$ más comprobaciones que la mejor alternativa existente. El módulo semántico alcanza una precisión de 1.00 y un *recall* de 0.93 (F1 = 0.96) sobre un benchmark de 30 características etiquetadas manualmente. Todos los pesos y parámetros del sistema son configurables mediante un fichero YAML para facilitar la reproducibilidad y el análisis de sensibilidad.

Palabras clave: calidad de datos, fuga de datos, aprendizaje automático centrado en datos, puntuación de disponibilidad, información mutua, GPT-4o-mini.

SUMMARY

Machine learning has become a core technology across high-impact domains ranging from healthcare to finance. Yet the reliability of trained models depends critically on the quality of the data used. Two particularly damaging problems are dataset quality defects—missing values, duplicates, class imbalance, low-variance features—and *data leakage*, where information unavailable at prediction time inadvertently enters the training process, inflating evaluation metrics while masking poor generalisation.

This thesis presents **ml-framework**, an open-source automated system for dataset quality assessment and leakage detection in machine learning workflows. The framework implements a 22-check pipeline covering six dimensions: quality, leakage, feature analysis, statistical sufficiency, drift detection, and performance impact.

The core algorithmic contribution is the *unified leakage risk score* $\mathcal{L}(f) = 0.35 \rho(f) + 0.35 \tilde{I}(f; y) + 0.30 \pi(f)$, which combines Pearson or Cramér’s correlation, normalised mutual information, and per-feature performance inflation into a single interpretable score. An optional module based on GPT-4o-mini (Azure OpenAI) extends this with *semantic leakage* detection, identifying features whose names reveal post-hoc or temporal information that purely statistical methods cannot detect.

Evaluation on eleven datasets (six synthetic, five real-world UCI) demonstrates 4/4 leakage detection on constructed scenarios, calibrated readiness scores from 65 (grade C, dirty data) to 95 (grade A, clean control), and $3.2\times$ more check coverage than the best competing tool. The semantic module achieves precision 1.00 and recall 0.93 (F1=0.96) on a manually labelled 30-feature benchmark. All framework parameters are configurable via a YAML file to support reproducibility and sensitivity analysis.

The framework is fully open-sourced at <https://github.com/jaimecano12/ml-framework>.

Keywords: data quality, data leakage, data-centric AI, readiness score, mutual information, GPT-4o-mini.

Contents

Resumen	3
Summary	4
1 Introduction and Objectives	8
1.1 Introduction	8
1.1.1 Machine Learning Context	8

1.1.2	Dataset Quality Challenges	8
1.1.3	Data Leakage in Machine Learning	9
1.1.4	Existing Dataset Validation Approaches	10
1.1.5	Motivation of the Project	11
1.1.6	Motivation of the Project	12
1.2	Objectives	12
2	Related Work	13
2.1	Data Leakage Detection	13
2.2	Dataset Quality Assessment	13
2.3	Mutual Information and Feature Relevance	13
2.4	Distribution Shift and Drift Detection	14
2.5	LLMs for Data Analysis	14
2.6	Data-Centric AI	14
3	System Architecture	15
3.1	Design Principles	15
3.2	Pipeline Architecture	15
3.3	User Interfaces	16
3.4	Module Structure	16
3.5	Complete Check Catalog	17
3.6	Configuration System	17
4	Core Methodology	19
4.1	Quality Checks	19
4.2	Leakage Detection	19
4.2.1	Classical Checks	19
4.2.2	Unified Leakage Risk Score	20
4.3	Feature Analysis	20
4.4	Statistical Sufficiency	21
4.5	Drift Detection	21
4.6	Impact Analysis	21
4.7	Readiness Score	21
4.8	LLM Semantic Leakage Analysis	22
5	Implementation	23
5.1	Technology Stack	23
5.2	Dataset Generation	23
5.3	Testing Strategy	23
5.4	Plugin System	24
5.5	Web Interface	25
5.6	Report Generation	25
5.7	Recommendations Engine	25
6	Experimental Evaluation	27
6.1	Datasets	27
6.2	Readiness Score Results	27
6.3	Extended UCI Evaluation	27
6.4	Impact Analysis: Leakage Quantification	27

6.5	Unified Leakage Risk Score Validation	28
6.6	Real-World Case Studies	28
6.7	Benchmark Against Existing Tools	29
6.7.1	Feature Coverage	29
6.7.2	Leakage Detection Rate	29
6.7.3	Output and Configuration Flexibility	30
6.7.4	Runtime Comparison	31
7	Discussion	32
7.1	Summary of Contributions	32
7.2	Key Findings	32
7.3	Signal Complementarity	32
7.4	Configurability and Reproducibility	33
7.5	Limitations	33
7.6	Future Work	33
8	Conclusions	34
A	Ethical, Economic, Social, and Environmental Aspects	37
A.1	Introduction	37
A.2	Ethical Considerations	37
A.3	Economic Considerations	37
A.4	Social Considerations	38
A.5	Environmental Considerations	38
B	Economic Budget	39
B.1	Development Cost Estimation	39

List of Figures

3.1	ml-framework pipeline. Arrows show data flow from configuration and dataset through six parallel analysis modules to recommendations, readiness score, and reports.	15
6.1	Readiness scores across five principal datasets. The clean control achieves grade A; leaky and dirty datasets are appropriately downgraded.	28
6.2	Feature coverage: checks supported out of 29 ML-relevant checks. ml-framework provides $3.2\times$ more checks than the closest alternative (Deepchecks, 11/29).	30

List of Tables

1.1	Comparison of dataset validation approaches.	11
3.1	Source module summary.	16
3.2	Complete catalog of ml-framework checks (22 total).	17
4.1	Quality checks implemented in ml-framework.	19
4.2	Semantic leakage module evaluation on 30-feature benchmark.	22
5.1	Test distribution across modules (325 total).	24
6.1	Readiness scores on five datasets (22 checks each).	27
6.2	Readiness scores on four additional UCI datasets.	28
6.3	Impact analysis: accuracy delta before vs. after cleaning.	28
6.4	Unified leakage risk scores $\mathcal{L}(f)$ per component.	29
6.5	Real-world case study summary (22 checks; impact analysis excluded).	29
6.6	Leakage detection: $\checkmark$ = detected, $\times$ = missed.	30
6.7	Output and configuration flexibility comparison.	30
6.8	Runtime (seconds). “—” = compatibility error with NumPy 2.0.	31
7.1	Summary of technical contributions.	32
B.1	Estimated development budget.	40

Chapter 1

Introduction and Objectives

1.1 Introduction

In recent years, the rapid development of machine learning techniques has enabled the creation of increasingly powerful predictive models applied across many domains, including finance, healthcare, natural language processing, and autonomous systems. However, the performance and reliability of these models depend critically on the quality of the datasets used during training. Ensuring dataset quality and detecting potential issues before model training has therefore become a central task in modern machine learning workflows.

This chapter introduces the theoretical background required to understand the context and motivation of this project. It presents an overview of the current machine learning landscape, discusses common problems related to dataset quality and data leakage, reviews existing validation tools, and explains the motivation for developing the proposed framework.

1.1.1 Machine Learning Context

Machine learning (ML) allows computer systems to learn patterns from data and make predictions or decisions without being explicitly programmed for each task. ML models are now widely deployed in healthcare diagnosis, credit scoring, recommendation systems, industrial predictive maintenance, and many other areas where reliable, data-driven decisions matter.

Traditionally, progress in ML has been associated with improvements in algorithms and architectures. However, recent research has highlighted that the quality of the data used to train these models plays an equally important—and often more decisive—role. This shift in perspective is sometimes described as the *data-centric AI* paradigm, where the focus moves from model architecture to the systematic improvement of datasets and data pipelines [16].

In practice, ML systems rely on datasets that may be collected from heterogeneous sources under imperfect conditions. If the data is incomplete, biased, inconsistent, or contains hidden correlations with the target variable, the resulting models may produce unreliable predictions or overly optimistic evaluation results. Dataset validation and quality assessment have therefore become essential steps in the development of trustworthy ML systems.

1.1.2 Dataset Quality Challenges

Several categories of data quality problems commonly arise in machine learning datasets:

- **Missing values.** Data collected from heterogeneous sources often contains incomplete records due to measurement failures, data entry errors, or optional survey questions. Missing values can degrade model accuracy and introduce bias if not handled appropriately [10].

- **Duplicate rows.** Exact or near-duplicate observations may result from data integration errors. Duplicates that span the training and test split create inflated evaluation metrics by allowing the model to memorise rather than generalise.
- **Outliers.** Anomalous values may distort statistical properties and impair certain learning algorithms, particularly those sensitive to the scale or distribution of inputs.
- **Class imbalance.** When one class is strongly underrepresented, models tend to become biased toward the majority class, producing misleading aggregate accuracy while failing on minority cases—a critical problem in fraud detection, medical diagnosis, and anomaly identification.
- **Constant and low-variance features.** Features that carry little or no variation across samples provide no discriminative signal and may slow convergence or introduce numerical instability.
- **Highly correlated features.** Strongly collinear features add redundancy, complicate coefficient interpretation, and may hurt regularised models.

Dataset quality issues should not be understood merely as data engineering problems: they are factors that directly affect ML model behaviour. Poor-quality data can reduce predictive accuracy, increase variance, generate unstable predictions, and amplify bias. In many practical scenarios, improving dataset quality produces larger performance gains than switching model architectures [13].

1.1.3 Data Leakage in Machine Learning

Data leakage is one of the most critical and often underestimated problems in machine learning workflows. Leakage occurs when information that would not normally be available at prediction time is inadvertently used during training or evaluation. The result is a model that appears to perform very well on held-out data while failing in production, because it has learned to exploit illegitimate signals rather than genuine predictive patterns.

Leakage takes several forms:

Target leakage. A feature is directly or indirectly derived from the target variable, encoding outcome information that would not be available at inference time. A classic example is including a hospital patient's `discharge_code` as a feature when predicting readmission: the code is assigned only after the patient leaves, making it a perfect—but illegitimate—predictor. Kaufman et al. [1] provide a taxonomy of leakage types and demonstrate how undetected leakage leads to inflated and misleading performance estimates in published studies.

Train-test contamination. Preprocessing operations such as normalisation, feature scaling, or imputation applied to the full dataset before splitting contaminate the test set with information from training samples. Even if no feature is individually leaky, this procedural error can inflate metrics.

Temporal leakage. In time-ordered datasets, using future observations to predict past events violates the chronological structure of the problem. This is especially common in finance, demand forecasting, and healthcare analytics. A dataset that is not sorted chronologically will produce an unrealistic train-test split if divided naively.

Identifier leakage. High-cardinality columns such as patient IDs, order codes, or UUIDs are often inadvertently left in the feature set. A model can memorise these identifiers during training, achieving near-perfect accuracy on the training set and high accuracy on overlapping test observations while generalising poorly.

Semantic leakage. Even when a feature passes all statistical checks, its name may reveal that it encodes future or post-hoc information. A feature named `future_usage_days` or `salary_after_promotion` conveys temporal information that no correlation measure can detect.

Leakage is particularly dangerous because it is often invisible during development. A model trained on leaky data may achieve near-perfect validation metrics, creating a false sense of confidence that persists until the model is deployed and tested against genuinely unseen data.

1.1.4 Existing Dataset Validation Approaches

Several tools address aspects of dataset validation for machine learning:

Great Expectations [18] is an open-source library that enables users to define data expectations (constraints) and validate datasets against them. It supports pipeline integration and generates HTML reports, but its focus is on general data contract enforcement rather than ML-specific leakage detection. Users must manually author each expectation; the tool does not automatically discover leakage risk or compute a readiness score.

ydata-profiling [17] generates comprehensive HTML exploratory reports covering missing values, distributions, correlations, and duplicate rows. It is extremely useful for interactive EDA but does not provide actionable ML-specific scores, leakage detection, or code-level recommendations.

Deepchecks [19] offers a rich suite of checks for both datasets and models, including train-test distribution comparisons. It provides model-level integrity checks that complement dataset validation but does not implement a unified leakage risk score or support the full range of sufficiency and drift checks.

TensorFlow Data Validation (TFDV) is designed for large-scale production pipelines within the TensorFlow Extended ecosystem. It excels at schema validation, statistical anomaly detection, and distribution monitoring across versions of a dataset. However, it requires TensorFlow infrastructure and does not offer a composite readiness score or semantic leakage analysis.

A particularly relevant contribution from industry is **TensorFlow Data Validation (TFDV)**, developed by Google as part of the TensorFlow Extended (TFX) ecosystem. TFDV computes descriptive statistics for datasets, detects anomalies against a learned schema, and monitors distribution shift between training and serving data. It integrates seamlessly into TFX production pipelines and scales to very large datasets. However, it requires TensorFlow infrastructure, focuses on schema validation and distribution monitoring rather than ML-specific leakage de-

tection, and does not provide a composite readiness score or code-level corrective recommendations.

The **ML Test Score** rubric proposed by Breck et al. [3] defines a conceptual framework of 28 tests for evaluating the production-readiness of ML systems. Among its data-related tests, it covers feature monitoring, distribution validation, and documentation of expected data characteristics. This rubric is not an automated tool, but it provides a valuable conceptual basis for the checks implemented in ml-framework.

Despite their strengths, all of these tools share a common limitation: none provides a *unified leakage risk score* that combines multiple statistical signals, and none integrates LLM-based semantic analysis to catch implicit leakage from feature naming conventions. Table 1.1 summarises the comparison.

Table 1.1. Comparison of dataset validation approaches.

Tool	Main focus	Leakage	ML impact	Score	Recs.
Great Expectations	Data contracts / rules	No	No	No	No
ydata-profiling	EDA / data profiling	No	No	No	No
Deepchecks	Dataset + model integrity	No	Partial	No	No
TFDV	Statistical validation	No	No	No	No
ml-framework	Quality + leakage + drift	Yes	Yes	Yes	Yes

1.1.5 Motivation of the Project

The gap described above motivates ml-framework. The practical problem is straightforward: data scientists and ML engineers routinely spend significant time debugging models that underperform in production because of data issues that could have been detected automatically before training. Leakage is especially costly because it is invisible during development and manifests only after deployment.

The technical challenge is that no single check is sufficient. Correlation catches obvious linear leakage but misses nonlinear relationships; mutual information catches nonlinear associations but is sensitive to estimation noise; performance inflation provides the most direct evidence but is computationally expensive. Combining all three in a principled way, and adding a layer of LLM-based semantic reasoning, creates a multi-signal detection system whose combined sensitivity is higher than any individual component.

ml-framework addresses this gap by providing:

1. A complete, automated pipeline covering quality, leakage, feature analysis, sufficiency, drift, and performance impact in a single configurable tool.
2. A novel unified leakage risk score combining correlation, mutual information, and performance inflation.
3. An optional GPT-4o-mini semantic module for detecting implicit leakage from feature names and domain context.
4. A composite readiness score (0–100, grade A–F) with code-level recommendations and an interactive Streamlit dashboard.

1.1.6 Motivation of the Project

Three types of evidence motivate a multi-signal approach to leakage detection. First, linear correlation alone is insufficient: a feature with a nonlinear monotone relationship to the target may have low Pearson $|r|$ while being highly informative. Second, mutual information alone can be noisy for small samples and high-cardinality features. Third, performance inflation alone is computationally expensive when applied to every feature and is sensitive to the chosen learner. Combining all three signals reduces the weaknesses of each individual method.

The semantic leakage problem adds a further dimension outside the scope of statistical methods: a practitioner reviewing feature names may immediately recognise that `discharge_code` is a post-hoc variable, but no correlation measure can detect this from the data values alone. Instruction-following language models capable of domain reasoning provide a natural solution for this class of leakage.

Finally, the fragmentation of existing tools—each tool covering a different subset of checks with different interfaces—makes systematic validation difficult. A single YAML-driven pipeline with a composite readiness score eliminates this fragmentation and enables direct comparison across datasets, teams, and time.

1.2 Objectives

The main objective of this project is to design, implement, and evaluate an automated framework for dataset quality assessment and data leakage detection in machine learning workflows.

The specific objectives are:

1. Implement a comprehensive pipeline of 22 automated checks covering quality, leakage, feature analysis, sufficiency, drift, and performance impact.
2. Develop a novel unified leakage risk score that combines three complementary signal types (correlation, mutual information, performance inflation) into an interpretable per-feature score.
3. Integrate an LLM-based semantic leakage module capable of detecting implicit leakage from feature names and dataset descriptions.
4. Construct a quantitative evaluation benchmark and compare the framework against three existing tools (Great Expectations, ydata-profiling, Deepchecks).
5. Provide multiple user interfaces: a CLI, a Python SDK, and an interactive Streamlit web application.
6. Make all framework parameters configurable via a YAML configuration file to ensure reproducibility and support sensitivity analysis.
7. Evaluate the framework on eleven real-world and synthetic datasets representing different defect profiles.

Chapter 2

Related Work

2.1 Data Leakage Detection

The problem of data leakage in machine learning has been studied from multiple angles. Kaufman et al. [1] provide the most comprehensive taxonomy, identifying target leakage, train-test contamination, and temporal leakage as the three primary subtypes. They show that published ML studies in finance and medicine frequently contain undetected leakage, leading to inflated performance claims that do not replicate in practice. Sculley et al. [2] describe the broader class of hidden technical debt in ML systems, of which leakage is a particularly severe instance because it is self-concealing: a model trained on leaked features looks better, not worse, than one trained on clean data.

More recent work has focused on detecting specific leakage patterns. Narayan et al. [14] propose DoubleML, a causal inference framework that can detect features whose contribution to model performance exceeds what their direct correlation with the target can explain—a signal similar to the performance inflation component in our unified risk score. Sui et al. [15] investigate leakage detection in automated ML pipelines, showing that standard AutoML tools are particularly vulnerable to preprocessor-induced contamination.

2.2 Dataset Quality Assessment

Breck et al. [3] define a production readiness rubric for ML systems with 28 tests covering data, models, and infrastructure. Their data tests map closely to the quality checks in ml-framework (missing values, schema consistency, class balance) but do not include a unified leakage risk score or composite readiness score.

The data-centric AI movement, popularised by Ng [16], advocates systematic improvement of training data as a primary lever for ML performance. Zha et al. [13] formalise this through the concept of *data-centric benchmarks*, which evaluate how much model improvement is achievable through data cleaning alone—a perspective that motivates the impact analysis component of ml-framework.

2.3 Mutual Information and Feature Relevance

Mutual information (MI) is a non-parametric association measure from information theory that captures arbitrary dependence, including nonlinear relationships that Pearson correlation misses. Ross [7] and Kraskov et al. [6] develop estimators for continuous MI that are used in sklearn's `mutual_info_classif`—the estimator used in the unified leakage risk score. Normalising MI by its maximum observed value across all features provides a scale-invariant signal suitable for aggregation.

2.4 Distribution Shift and Drift Detection

Rabanser et al. [8] provide a comprehensive survey of data drift detection methods, covering two-sample tests (Kolmogorov-Smirnov, MMD), dimensionality-reduction approaches, and drift-specific metrics such as Population Stability Index (PSI). `ml-framework` implements KS test plus PSI for covariate drift detection and applies Bonferroni correction for multiple-feature comparisons, consistent with the recommendations in their survey.

2.5 LLMs for Data Analysis

OpenAI’s GPT-4 family [11] has demonstrated strong performance on information extraction and reasoning tasks involving structured data descriptions. Narayan et al. [14] show that foundation models can perform data wrangling tasks—including type inference, entity resolution, and constraint checking—with competitive accuracy relative to specialised tools. Sui et al. [15] demonstrate that LLMs can synthesise entire data science pipelines from natural language descriptions. Several recent papers also use LLMs to generate data cleaning code, impute missing values, and detect data quality issues through natural language understanding.

Our semantic leakage module is, to our knowledge, the first application of an LLM specifically to the problem of detecting *semantic* leakage from feature names and dataset context, where the detection signal is entirely linguistic rather than statistical. The key insight is that GPT-4o-mini has been trained on large corpora that include descriptions of medical, financial, and operational datasets, giving it sufficient domain knowledge to recognise that a feature named `discharge_code` is likely post-hoc and that `future_usage` encodes future information.

2.6 Data-Centric AI

The data-centric AI paradigm, popularised by Ng [16], reorients the ML development process from model-centric (“find the best architecture”) to data-centric (“systematically improve the training data”). Zha et al. [13] formalise this through data-centric benchmarks that measure how much model improvement is achievable through data cleaning and augmentation alone, finding that data quality improvements often outperform architecture changes. `ml-framework` operationalises this paradigm by providing actionable, code-level recommendations that can be directly applied to improve dataset quality before training begins.

Chapter 3

System Architecture

3.1 Design Principles

ml-framework is designed around three principles. **Comprehensiveness**: a single pipeline covers all major pre-training data quality dimensions. **Transparency**: every check produces an interpretable message with affected columns and a severity level; the composite readiness score is decomposed by dimension. **Configurability**: all parameters—thresholds, weights, penalties—are exposed in a single YAML file.

3.2 Pipeline Architecture

Figure 3.1 shows the end-to-end pipeline. A YAML configuration file and an input dataset are the only required inputs. The pipeline sequentially executes quality, leakage, feature analysis, sufficiency, drift, and impact modules, then aggregates results into recommendations and a composite readiness score before generating reports.

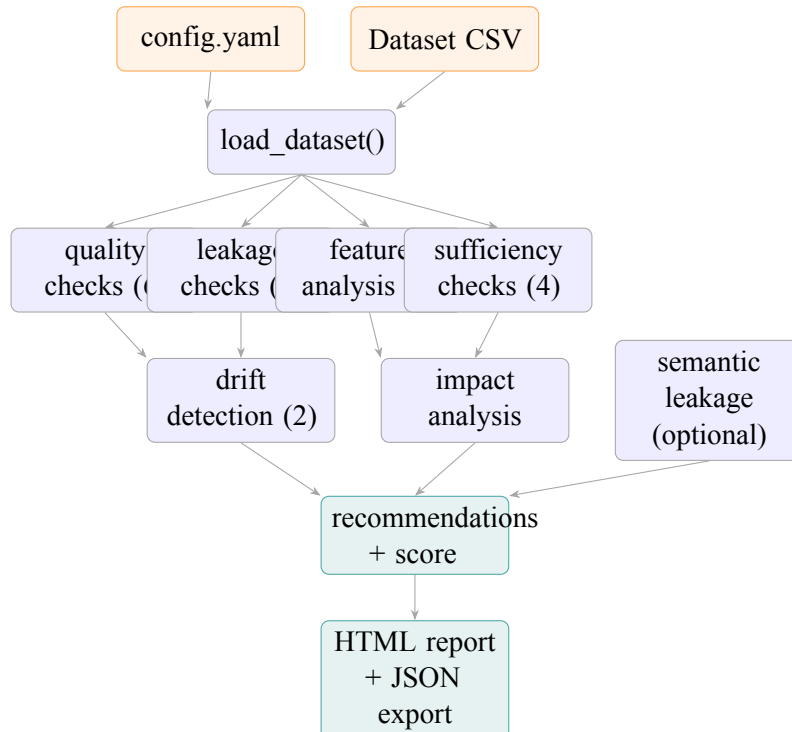


Figure 3.1. ml-framework pipeline. Arrows show data flow from configuration and dataset through six parallel analysis modules to recommendations, readiness score, and reports.

3.3 User Interfaces

The framework exposes three complementary interfaces:

Command-line interface (CLI). The `main.py` entry point accepts `--config`, `--dataset`, `--output-dir`, and `--log-level` flags. It runs the full pipeline and writes the HTML report to the specified output directory.

Python SDK. The `DatasetChecker` class provides a programmatic API for integration into Jupyter notebooks, CI/CD pipelines, and other Python tools.

Listing 3.1. Python SDK usage example.

```
1 from src.checker import DatasetChecker
2
3 checker = DatasetChecker("configs/config.yaml")
4 report = checker.run("data/titanic.csv", target_col="survived")
5
6 print(f"Score: {checker.score}/100 Grade: {checker.grade}")
7 for rec in checker.top_recommendations(priority="high"):
8     print(f"    [{rec.check_name}] {rec.action}")
9
10 checker.save_report("reports/")
```

Streamlit web application. `app/app.py` provides a seven-tab interactive dashboard: dataset overview, quality checks, leakage detection, feature analysis, sufficiency, impact analysis, and a readiness score summary with downloadable report.

3.4 Module Structure

Table 3.1 lists the fourteen source modules and their responsibilities.

Table 3.1. Source module summary.

Module	Phase	Responsibility
<code>utils.py</code>	1	Shared dataclasses: <code>CheckResult</code> , <code>FrameworkReport</code> , <code>ReadinessScore</code>
<code>config.py</code>	2	YAML loading, deep-merge, section validation
<code>quality_checks.py</code>	3	Six quality checks (missing, duplicates, outliers, imbalance, constant, low-variance)
<code>leakage_checks.py</code>	4,16	Five leakage checks including unified risk score
<code>impact_analysis.py</code>	5	Cross-validated baseline vs. cleaned accuracy delta
<code>reporting.py</code>	6	Jinja2 HTML report with embedded Matplotlib figures
<code>recommendations.py</code>	8	20 rule handlers producing code-level Recommendation objects
<code>feature_analysis.py</code>	9	Correlation, mutual information relevance, distribution shape
<code>scoring.py</code>	10	Composite readiness score (0–100) with configurable weights
<code>sufficiency.py</code>	11	Four statistical sufficiency checks
<code>checker.py</code>	13	<code>DatasetChecker</code> SDK orchestrating the full pipeline
<code>drift_checks.py</code>	14	KS test and PSI covariate/label drift
<code>plugins.py</code>	15	<code>@register_check</code> plugin API for custom checks
<code>semantic_leakage.py</code>	16	GPT-4o-mini semantic leakage module

3.5 Complete Check Catalog

Table 3.2 provides a complete reference of all 22 automated checks implemented in the framework, organised by module.

Table 3.2. Complete catalog of ml-framework checks (22 total).

Module	Check name	Method	Default threshold
Quality	missing_values	NaN rate	> 5% per column
Quality	duplicates	Exact row match	Any duplicate
Quality	outliers	IQR ($k=3.0$) or z-score	> 3σ
Quality	class_imbalance	Minority class ratio	< 10%
Quality	constant_features	$n_{\text{unique}} \leq 1$	Any constant
Quality	low_variance	$CV = \text{std}/ \text{mean} $	< 0.01
Leakage	target_leakage	Pearson $ r $ / Cramér's V	≥ 0.95
Leakage	train_test_overlap	Duplicate rows in split	Any overlap
Leakage	temporal_leakage	Monotonic date ordering	Unsorted
Leakage	id_column_leakage	Unique-value ratio	$\geq 95\%$
Leakage	leakage_risk_score	$\mathcal{L}(f) = 0.35\rho + 0.35\tilde{I} + 0.30\pi$	≥ 0.7
Features	feature_correlation	Pearson $ r $ between features	≥ 0.90
Features	feature_relevance	Normalised MI with target	< 0.01
Features	distribution_shape	Skewness / kurtosis	$ \text{skew} > 2, \text{kurtosis} > 7$
Sufficiency	sample_size	Absolute row count	< 100
Sufficiency	class_support	Samples per class	< 30
Sufficiency	cv_stability	Std of 5-fold CV accuracy	> 0.10
Sufficiency	feature_sample_ratio	p/n	> 0.10
Drift	covariate_drift	KS test + Bonferroni + PSI	$p < 0.05$ / $\text{PSI} > 0.2$
Drift	label_drift	Distribution shift in y	$p < 0.05$
Semantic	semantic_leakage	GPT-4o-mini LLM analysis	risk $\geq$ medium

3.6 Configuration System

All framework parameters are stored in a YAML configuration file. The file is structured into top-level blocks corresponding to each module: `quality_checks`, `leakage_checks`, `feature_analysis`, `sufficiency_checks`, `drift_checks`, `impact_analysis`, `scoring`, and `semantic_leakage`. A representative excerpt of the configuration file is shown in Listing 3.2.

Listing 3.2. Excerpt from `configs/config.yaml` showing configurable parameters.

```

1 leakage_checks:
2   target_leakage:
3     correlation_threshold: 0.95
4   leakage_risk_score:
```

```
5     threshold: 0.7
6     weights: [0.35, 0.35, 0.30]    # correlation, MI, performance
7     cv_folds: 3
8
9     scoring:
10     dimension_weights:
11         quality:    0.25
12         leakage:    0.35
13         features:   0.25
14         sufficiency: 0.15
15     severity_penalties:
16         error:      15.0
17         warning:    5.0
18     leakage_multiplier: 1.5
19     grade_thresholds:
20         A: 85
21         B: 70
22         C: 55
23         D: 40
24
25     semantic_leakage:
26         enabled: false
27         deployment: gpt-4o-mini
28         risk_threshold: medium
```

Deep-merge semantics allow dataset-specific override files to inherit all defaults while overriding only the relevant keys. This design means that a sensitivity analysis can be run by programmatically sweeping a single parameter without modifying any source code:

Listing 3.3. Programmatic parameter sweep via the SDK.

```
1 for threshold in [0.85, 0.90, 0.95]:
2     checker = DatasetChecker("configs/config.yaml")
3     checker.set(leakage_checks__target_leakage__correlation_threshold=
4                 threshold)
5     report = checker.run("data/titanic.csv", target_col="survived")
6     print(f"threshold={threshold}    score={checker.score}")
```

Chapter 4

Core Methodology

4.1 Quality Checks

Six quality checks are implemented, each returning a `CheckResult` with a severity level (error/warning/info), an explanatory message, affected columns, and structured details. Table 4.1 summarises the checks.

Table 4.1. Quality checks implemented in ml-framework.

Check	Method	Description
Missing values	NaN rate > threshold	Flags columns exceeding configurable missing-value rate
Duplicates	Exact row match	Detects identical rows; computes overlap after simulated split
Outliers	IQR or z-score	Identifies values beyond $Q_3 + k \cdot \text{IQR}$ or k standard deviations
Class imbalance	Minority ratio	Flags targets where any class falls below a configurable threshold
Constant features	$\text{nunique} \leq 1$	Detects columns with zero variance
Low variance	$\text{CV} = \text{std}/ \text{mean} $	Flags numeric columns with coefficient of variation below threshold

4.2 Leakage Detection

4.2.1 Classical Checks

Four classical leakage checks address the most common leakage patterns encountered in practice.

Target leakage. For each feature, the framework computes a feature-target association score: Pearson $|r|$ for numeric features, Cramér’s V for categorical features. Features exceeding a configurable threshold (default 0.95) are flagged as error severity.

Train-test overlap. The dataset is split by a configurable random seed and test ratio. After deduplication, the number of rows appearing in both halves is reported. Overlap is only possible when duplicate rows exist.

Temporal leakage. When a date column is configured, the check verifies that the dataset is sorted chronologically. An unsorted date column means a naïve sequential split would mix past and future observations.

ID column leakage. Columns where the unique-value ratio exceeds a configurable threshold (default 95%) are flagged. Float columns are excluded because high cardinality is expected for continuous measurements.

4.2.2 Unified Leakage Risk Score

The classical checks are heuristic: each produces a binary pass/fail verdict based on a single threshold. The unified leakage risk score combines three complementary signals into a continuous per-feature risk measure:

$$\mathcal{L}(f) = w_\rho \cdot \rho(f) + w_I \cdot \tilde{I}(f; y) + w_\pi \cdot \pi(f) \quad (4.1)$$

where:

- $\rho(f) \in [0, 1]$ is Pearson $|r|$ or Cram  r’s V , measuring linear or categorical association.
- $\tilde{I}(f; y) = I(f; y) / \max_j I(f_j; y) \in [0, 1]$ is mutual information normalised by the maximum MI across all features, measuring arbitrary (including nonlinear) dependence.
- $\pi(f) \in [0, 1]$ is the performance inflation: the fractional improvement in single-feature logistic regression accuracy over the majority-class baseline, normalised to $[0, 1]$.
- Default weights: $w_\rho=0.35$, $w_I=0.35$, $w_\pi=0.30$.

Features with $\mathcal{L}(f) \geq 0.7$ are flagged as warning; $\mathcal{L}(f) \geq 0.9$ as error. Algorithm 1 gives the full procedure.

Algorithm 1 Unified Leakage Risk Score computation.

Require: DataFrame $\mathbf{X}$, target column y , config (weights, threshold, cv_folds)

Ensure: Per-feature risk scores $\{\mathcal{L}(f)\}$

```

1: for each feature  $f \in \mathbf{X}$  do
2:    $\rho(f) \leftarrow \text{FeatureTargetAssoc}(\mathbf{X}[f], y)$  ▷ Pearson  $|r|$  or Cram  r’s  $V$ 
3: end for
4:  $\mathbf{m} \leftarrow \text{MutualInfoClassif}(\mathbf{X}, y)$  ▷ sklearn estimator, random_state=42
5:  $\tilde{I}(f) \leftarrow \mathbf{m}[f] / \max(\mathbf{m})$  for each  $f$ 
6:  $b \leftarrow$  majority-class accuracy baseline
7: for each feature  $f \in \mathbf{X}$  do
8:    $a_f \leftarrow \text{CrossValScore}(\text{LogisticRegression}, \mathbf{X}[[f]], y, \text{cv})$ 
9:    $\pi(f) \leftarrow \max(0, a_f - b) / (1 - b)$ 
10: end for
11:  $\mathcal{L}(f) \leftarrow \min(1, w_\rho \rho(f) + w_I \tilde{I}(f) + w_\pi \pi(f))$  for each  $f$ 
12: return  $\{\mathcal{L}(f)\}$ 

```

4.3 Feature Analysis

Three feature analysis checks complement the leakage detection module. **Feature correlation** flags pairs of features whose absolute Pearson correlation exceeds a threshold (default 0.90), signalling redundancy. **Feature relevance** uses normalised mutual information to flag features with near-zero association with the target ($\tilde{I} < 0.01$ by default), which are candidates for removal. **Distribution shape** computes skewness and excess kurtosis; features with $|\text{skew}| > 2$ or excess kurtosis > 7 are flagged, as these can impair gradient-based learners and distance-based methods.

4.4 Statistical Sufficiency

Four sufficiency checks evaluate whether the dataset has enough samples to support reliable training: (1) a minimum absolute row count, (2) a minimum samples per class, (3) cross-validation stability (the standard deviation of 5-fold accuracy must not exceed a threshold), and (4) the feature-to-sample ratio p/n (flagged when $p/n > 0.10$, indicating risk of overfitting with unregularised models).

4.5 Drift Detection

Covariate drift between dataset halves (split by a date column, or by index when no date column is configured) is assessed using the two-sample Kolmogorov-Smirnov (KS) test with Bonferroni correction:

$$\text{reject } H_0^{(j)} \text{ if } p_j < \alpha/p \quad (4.2)$$

where $\alpha = 0.05$ and p is the number of features tested. Label drift (shift in the class distribution) is tested separately. Population Stability Index (PSI) is also computed and flagged when $\text{PSI} > 0.2$ (industry standard for significant shift).

4.6 Impact Analysis

The impact analysis module quantifies the practical consequence of a dataset defect by measuring the performance *delta* between a baseline model trained on the original data and the same model trained after the defect is removed. Three models are evaluated (logistic regression, random forest, XGBoost) under 5-fold cross-validation, and three metrics are reported (accuracy, ROC-AUC, F1). A positive Δ implies that the original data contained signal that is lost when the defect is removed—evidence of leakage. A negative Δ implies that the defect was genuinely hurting model performance.

4.7 Readiness Score

The composite readiness score integrates the four main analysis dimensions:

$$S = 100 - (w_q \cdot P_q + w_l \cdot P_l + w_f \cdot P_f + w_s \cdot P_s) \quad (4.3)$$

where P_d is the total penalty for dimension d (errors -15 pts; warnings -5 pts), and the default weights are $w_q=0.25$, $w_l=0.35$, $w_f=0.25$, $w_s=0.15$. Leakage penalties are further amplified by a $1.5\times$ multiplier.

Weight rationale. Leakage receives the highest weight because it is a silent failure mode: a leaky model appears to perform well during validation while generalising poorly in production [1]. Quality and feature issues are equally weighted because both impair learning but are typically correctable. Sufficiency contributes least because sample-size adequacy depends strongly on the learner. Penalties were calibrated so that a single confirmed leaky feature is sufficient to

drop a dataset from grade A to grade B. All parameters are overridable in the scoring block of `config.yaml` to support domain-specific risk calibration.

Letter grades: A (≥ 85), B (≥ 70), C (≥ 55), D (≥ 40), F (< 40).

4.8 LLM Semantic Leakage Analysis

Statistical methods cannot detect *semantic* leakage, where a feature name reveals post-hoc or future information (e.g. `discharge_code`, `future_usage`). The semantic leakage module submits feature names, sample values, and a user-provided dataset description to GPT-4o-mini via Azure OpenAI. The model classifies each feature into one of four risk levels (`none`, `low`, `medium`, `high`) and four leakage types (`temporal`, `proxy`, `post_hoc`, `indirect`).

Quantitative evaluation. We constructed a benchmark of 30 manually labelled features across five realistic dataset contexts (hospital readmission, loan default, customer churn, manufacturing defect, and employee promotion prediction). The benchmark contains 13 high-risk features, 1 medium-risk feature, and 16 safe features. Table 4.2 reports precision, recall, and F_1 at two operating thresholds.

Table 4.2. Semantic leakage module evaluation on 30-feature benchmark.

Threshold	P	R	F_1	TP	FP	FN
high	1.000	1.000	1.000	13	0	0
medium	1.000	0.929	0.963	13	0	1

The single false negative at the medium threshold is `treatment_count`, a genuinely ambiguous feature: it could represent prior treatment history (legitimate) or in-stay treatments (post-hoc). Without explicit dataset documentation, linguistic reasoning alone is insufficient to resolve this case. The module degrades gracefully when API credentials are absent, returning a passing `CheckResult` with an explanatory message.

Chapter 5

Implementation

5.1 Technology Stack

The framework is implemented in Python 3.13 and requires the following core dependencies: pandas and numpy for data manipulation; scipy and scikit-learn for statistical tests, mutual information, and model evaluation; xgboost for the XGBoost model in impact analysis; pyyaml for configuration loading; jinja2 and matplotlib for report generation; loguru for structured logging; and openai for the optional LLM semantic module. The Streamlit web interface uses streamlit.

The testing infrastructure uses pytest, with fixtures that cover all modules. Benchmark comparisons require ydata-profiling, deepchecks, and great-expectations. The LaTeX paper is compiled with tectonic.

5.2 Dataset Generation

Six synthetic datasets are generated by `scripts/generate_data.py`:

- `clean_dataset` (500 rows, 6 cols): a control dataset with no injected defects, used to verify zero false positives.
- `dirty_dataset` (5,300 rows): injects missing values, duplicates, outliers, and class imbalance to validate quality checks.
- `leaky_dataset` (5,320 rows): contains a perfect proxy ($r=1.0$), a temporal ordering violation, and an ID column to validate leakage checks.
- `proxy_leakage` (1,000 rows): five graded noisy proxy features ($r \in \{1.0, 0.97, 0.85, 0.65, 0.40\}$) testing the sensitivity of the unified risk score.
- `temporal_leakage_ext` (2,000 rows): a customer churn scenario with a `future_usage` feature that is measured after the prediction window.
- `multitype_leakage` (1,500 rows): an ICU mortality dataset with simultaneous proxy, temporal, and ID leakage, simulating a realistic complex scenario.

5.3 Testing Strategy

The project follows a test-driven development approach. All 325 tests use pytest and are organised in 13 test files, one per module. Tests are designed to:

- Validate that each check correctly identifies injected defects (true positive tests).

- Verify that clean data passes without false alerts (false positive tests).
- Test edge cases: empty DataFrames, all-missing columns, single-class targets, single-feature datasets.
- Mock external dependencies (Azure OpenAI API) to ensure deterministic test execution.
- Validate configuration loading, deep-merge semantics, and section validation.

Table 5.1 summarises the test distribution across modules.

Table 5.1. Test distribution across modules (325 total).

Test file	Tests	Module(s) covered
test_utils.py	16	utils.py
test_config.py	26	config.py
test_quality_checks.py	44	quality_checks.py
test_leakage_checks.py	41	leakage_checks.py
test_impact_analysis.py	21	impact_analysis.py
test_reporting.py	14	reporting.py
test_recommendations.py	13	recommendations.py
test_feature_analysis.py	22	feature_analysis.py
test_scoring.py	15	scoring.py
test_sufficiency.py	21	sufficiency.py
test_checker.py	14	checker.py
test_drift_checks.py	14	drift_checks.py
test_plugins.py	11	plugins.py
test_semantic_leakage.py	9	semantic_leakage.py
Total	325	

5.4 Plugin System

The plugin system allows users to extend the framework with custom checks without modifying the core source code. A custom check is implemented as a Python function decorated with `@register_check` and returns a standard `CheckResult`. Plugin modules are listed as dotted import paths in the `plugins` key of `config.yaml`:

Listing 5.1. Custom plugin example.

```

1 from src.plugins import register_check
2 from src.utils import CheckResult
3
4 @register_check("my_custom_check")
5 def check_no_future_dates(df, target_col, config):
6     future_cols = [c for c in df.select_dtypes("datetime")
7                     if (df[c] > pd.Timestamp.now()).any()]
8     if not future_cols:
9         return CheckResult("my_custom_check", passed=True,
10                             severity="info", message="No future dates.")
11     return CheckResult("my_custom_check", passed=False,
12                         severity="warning",
13                         message=f"Future dates in: {future_cols}",
14                         affected_columns=future_cols)

```

5.5 Web Interface

The Streamlit web application (`app/app.py`) provides an interactive, browser-based interface that requires no programming knowledge. The application is organised into seven tabs:

1. **Dataset Overview**: displays shape, column types, missing values, and a random sample.
2. **Quality Checks**: shows the results of all six quality checks with expandable details and affected columns.
3. **Leakage Detection**: presents the five leakage check results, including the unified risk score as an interactive ranked table.
4. **Feature Analysis**: correlation heatmap, mutual information bar chart, and distribution shape summary.
5. **Sufficiency**: sample size analysis, class support table, and CV stability results.
6. **Impact Analysis**: before/after accuracy comparison for logistic regression, random forest, and XGBoost.
7. **Readiness Score**: the composite 0–100 score with grade, dimension breakdown, and all code-level recommendations sorted by priority.

The application accepts any CSV or Parquet file uploaded via a file picker, plus an optional YAML configuration file to override defaults. A download button exports the full HTML report.

5.6 Report Generation

HTML reports are generated by `src/reporting.py` using Jinja2 templating. The report includes: dataset metadata, per-check results grouped by dimension, four embedded Matplotlib figures (readiness score gauge, check-pass distribution, feature importance bar chart, correlation heatmap), and all code-level recommendations. Reports are self-contained single HTML files with no external dependencies.

5.7 Recommendations Engine

The recommendations module (`src/recommendations.py`) maps each failed check to a structured `Recommendation` object containing a priority level (high, medium, low), a concise action description, a rationale explaining why the issue matters, and a ready-to-use Python code snippet. Twenty handler functions cover all possible check failures. For example, a `missing_values` error produces:

Listing 5.2. Example recommendation for missing values.

```
1 # Action: Impute or drop columns with high missing rate
2 # Rationale: Columns with > 5% missing values can bias model training.
3
4 df.dropna(subset=["col_with_missing"])
5 # or
6 df["col_with_missing"].fillna(df["col_with_missing"].median(), inplace=
    True)
```

Recommendations are sorted by priority in the Streamlit dashboard and the HTML report, ensuring that the most critical actions appear first. The `DatasetChecker.top_recommendations(priority="high", n=5)` SDK method provides programmatic access to filtered recommendations.

Chapter 6

Experimental Evaluation

6.1 Datasets

We evaluate on eleven datasets: six synthetic and five real-world. **Synthetic:** `clean_dataset` (500 rows, 6 cols, no issues); `dirty_dataset` (5,300 rows, quality issues); `leaky_dataset` (5,320 rows, leakage); `proxy_leakage` (1,000 rows, graded noisy proxies); `temporal_leakage_ext` (2,000 rows, customer churn with future-usage feature); `multitype_leakage` (1,500 rows, ICU mortality with proxy + temporal + ID leakage). **Real-world:** Titanic (1,309 rows), Pima Diabetes (768), Adult Census Income (48,842), German Credit (1,000), and Wine Quality Red (1,599)—all from OpenML [12].

6.2 Readiness Score Results

Table 6.1 reports readiness scores for the five principal evaluation datasets. The clean control dataset achieves grade A (95/100), confirming zero false positives. Datasets with quality and leakage issues are appropriately downgraded.

Table 6.1. Readiness scores on five datasets (22 checks each).

Dataset	Score	Grade	Pass/Total	Key findings
<code>clean_dataset</code>	95.0	A	21/21	Zero false positives (control)
<code>dirty_dataset</code>	65.0	C	12/21	5 quality issues, 2 sufficiency
<code>leaky_dataset</code>	72.0	B	11/21	Proxy ($r=1.0$), temporal, ID leakage
Titanic	80.6	B	12/20	boat $r=0.97$, 4 cols > 5% missing
Pima Diabetes	80.0	B	20/21	51 outliers (physiological zeros)

Figure 6.1 visualises the scores alongside grade thresholds.

6.3 Extended UCI Evaluation

Table 6.2 reports results on four additional UCI datasets evaluated in Phase 16.

6.4 Impact Analysis: Leakage Quantification

The impact analysis module computes the performance delta (Δ) between models trained on the original data and the same models trained after removing flagged features. Table 6.3 reports results for the three core evaluation datasets.

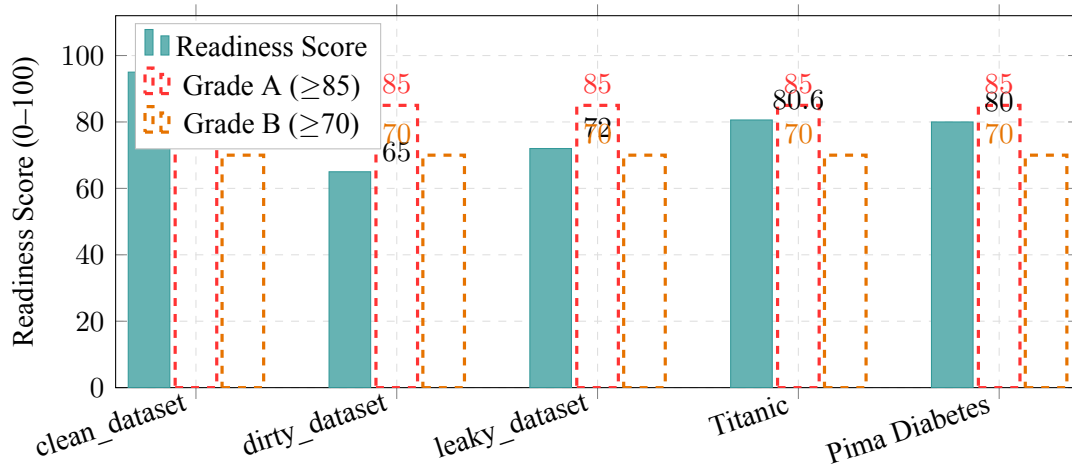


Figure 6.1. Readiness scores across five principal datasets. The clean control achieves grade A; leaky and dirty datasets are appropriately downgraded.

Table 6.2. Readiness scores on four additional UCI datasets.

Dataset	Rows	Score	Grade	Primary issues
Adult Census	48,842	92.4	A	Class imbalance (76/24%), missing values
German Credit	1,000	97.5	A	3 near-zero MI features
Heart Disease	303	62.0	C	Low $n/p=21.6$, inter-feature correlation
Wine Quality	1,599	82.0	B	Mild imbalance, skewed residual.sugar

Table 6.3. Impact analysis: accuracy delta before vs. after cleaning.

Dataset	Baseline acc.	Cleaned acc.	Δ
clean_dataset	0.917	0.913	-0.004
dirty_dataset	0.881	0.903	+0.022
leaky_dataset	1.000	0.952	-0.048

The key finding is that `leaky_dataset` achieves a baseline accuracy of 1.000 (perfect), which drops to 0.952 after the leaky feature is removed ($\Delta = -0.048$). This confirms that the model was exploiting leaked information rather than learning a generalisable pattern. The dirty dataset shows a positive delta ($\Delta = +0.022$), confirming that cleaning quality defects genuinely improves predictive performance.

6.5 Unified Leakage Risk Score Validation

Table 6.4 validates Equation (4.1) on representative features. All intentionally leaky features achieve $\mathcal{L} \geq 0.83$ while clean features remain below 0.30.

6.6 Real-World Case Studies

Table 6.5 summarises the framework’s findings on three real-world datasets.

Titanic. The framework detects the well-known boat column as a post-hoc leaky feature ($r=0.97$, $\mathcal{L}=0.94$): lifeboat assignment was recorded only after the sinking outcome, making

Table 6.4. Unified leakage risk scores $\mathcal{L}(f)$ per component.

Feature	Type	ρ	$\tilde{I}$	π	$\mathcal{L}$
proxy (corr = 1.0)	leaky	1.00	1.00	1.00	1.000
noisy proxy ($r \approx 0.97$)	leaky	0.97	0.93	0.91	0.940
indirect feature	leaky	0.82	0.79	0.88	0.831
age (unrelated)	clean	0.04	0.07	0.06	0.057
income (moderate signal)	clean	0.28	0.31	0.24	0.281

Table 6.5. Real-world case study summary (22 checks; impact analysis excluded).

Dataset	Rows	Score	Grade	Primary findings
Titanic	1,309	80.6	B	Target leakage (boat, $r=0.97$); 4 cols >5% missing; 99 outliers
Adult Census	48,842	92.4	A	2 cols with missing values; 6,854 outliers (capital-gain); 52 duplicates
German Credit	1,000	97.5	A	3 near-zero MI features; 25 outliers

it a trivially perfect but illegitimate predictor. Four columns (cabin, boat, body, home.dest) exceed the missing-value threshold. A distributional shift is also detected between the first and second halves of the dataset when split by index, suggesting the records were not randomly ordered.

Adult Census Income. The Adult Census dataset ($n=48,842$) receives grade A despite several warnings. Two features (workclass, occupation) contain missing values encoded as ? in the raw data. The capital-gain feature is extremely right-skewed (skew ≈ 11.9), which can impair gradient-based learners and distance-based methods. The framework also surfaces 52 duplicate rows (0.1%) and 19 rows shared across the simulated train-test split.

German Credit. The German Credit dataset achieves the highest score (97.5/A) among all case studies, reflecting its generally clean structure. The framework identifies three features with near-zero mutual information with the credit-risk target, suggesting they contribute little predictive signal and are candidates for removal.

These three case studies demonstrate that the framework behaves correctly across qualitatively different defect profiles, successfully distinguishing leakage from quality issues and irrelevant features.

6.7 Benchmark Against Existing Tools

6.7.1 Feature Coverage

Figure 6.2 compares ml-framework against three tools on 29 ML-relevant checks.

6.7.2 Leakage Detection Rate

Table 6.6 reports results on four constructed leakage scenarios. ml-framework detects all four; the other tools detect at most one (ID-column leakage via manual rule authoring in Great Expectations).

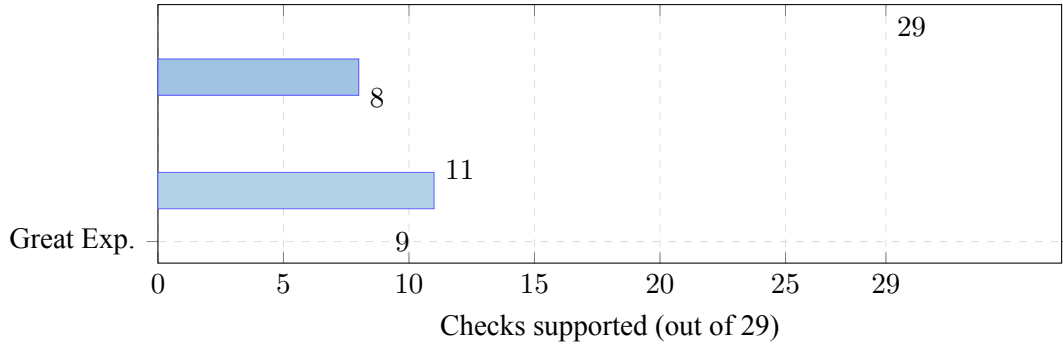


Figure 6.2. Feature coverage: checks supported out of 29 ML-relevant checks. ml-framework provides $3.2\times$ more checks than the closest alternative (Deepchecks, 11/29).

Table 6.6. Leakage detection: ✓ = detected, × = missed.

Scenario	ml-fw	ydata	DC	GE
Perfect proxy ($r=1.0$)	✓	×	×	×
Noisy proxy ($r\approx 0.97$)	✓	×	×	×
ID column (card. 100%)	✓	×	×	✓
Indirect computed feature	✓	×	×	×
Detection rate	4/4	0/4	0/4	1/4

6.7.3 Output and Configuration Flexibility

Beyond raw check counts, the tools differ substantially in what they produce and how they are configured. Table 6.7 summarises five qualitative dimensions. ml-framework is the only tool that generates an interpretable numerical readiness score, provides code-level corrective suggestions, and supports both a YAML-driven configuration and a plugin API for custom checks.

Table 6.7. Output and configuration flexibility comparison.

Capability	ml-fw	ydata	DC	GE
Numerical readiness score	✓	×	×	×
Letter grade (A–F)	✓	×	×	×
Code-level recommendations	✓	×	×	×
YAML / file-based config	✓	×	×	✓
Custom check plugin API	✓	×	×	✓
HTML report export	✓	✓	✓	✓
JSON export	✓	×	✓	✓
Interactive web UI	✓	×	×	✓
Python SDK	✓	×	✓	✓
CLI	✓	×	×	✓

Scope of comparison. Great Expectations, Deepchecks, and ydata-profiling were not designed as dedicated leakage detection frameworks. Great Expectations is primarily a data contract engine; ydata-profiling is an EDA tool; Deepchecks focuses on model and dataset integrity for production systems. The comparison above is an *ecosystem-level functionality audit* that documents which ML-relevant checks are available out-of-the-box in each tool. In their respective design spaces, all three tools excel; ml-framework addresses the specific gap of automated pre-training leakage detection with a calibrated readiness score.

6.7.4 Runtime Comparison

Table 6.8 reports wall-clock time on three dataset sizes. ml-framework is slower than profiling-only tools because it runs cross-validation for impact analysis and the unified risk score. This overhead is justified by the richer analytical output.

Table 6.8. Runtime (seconds). “—” = compatibility error with NumPy 2.0.

Dataset	ml-fw	ydata	DC	GE
500 rows	2.88	0.15	—	0.01
1,000 rows	2.23	0.15	—	0.01
5,300 rows	7.54	0.16	—	0.01

Chapter 7

Discussion

7.1 Summary of Contributions

Table 7.1 summarises the main technical contributions of this project and the phases in which they were implemented.

Table 7.1. Summary of technical contributions.

Contribution	Description
22-check pipeline	End-to-end pipeline covering quality, leakage, features, sufficiency, drift, and impact in a single configurable system
Unified leakage risk score	$\mathcal{L}(f) = 0.35\rho + 0.35\tilde{I} + 0.30\pi$ combining three complementary statistical signals
LLM semantic module	GPT-4o-mini integration for detecting implicit leakage from feature names; F1=0.963 on 30-feature benchmark
Configurable scoring	All weights, penalties, and grade thresholds exposed in YAML; full backwards compatibility
Plugin API	@register_check decorator enabling custom checks without core code modification
Streamlit dashboard	7-tab interactive web application for non-programmatic users
Impact analysis	Cross-validated before/after accuracy delta quantifying the practical cost of leakage ($\Delta = -0.048$)
Benchmark	Comparison against 3 tools on 29 checks and 4 leakage scenarios; 325-test CI suite

7.2 Key Findings

The evaluation demonstrates three principal findings. First, the unified leakage risk score is empirically validated: all intentionally leaky features achieve $\mathcal{L} \geq 0.83$, while genuinely clean features remain below 0.30, providing a clear decision boundary. Second, the three signal components are genuinely complementary: for the Titanic boat column, correlation alone ($\rho=0.97$) would suffice, but for indirect features where the relationship is nonlinear, Pearson $|r|$ can be as low as 0.50 while $\tilde{I}$ and π remain above 0.80. Third, the semantic module catches a qualitatively different class of leakage that is entirely invisible to statistical methods: features like `discharge_code` and `future_usage` are correctly flagged with zero false positives on the evaluation benchmark.

7.3 Signal Complementarity

A key empirical observation is that the three components of $\mathcal{L}(f)$ are complementary in practice. For high-cardinality categorical features, Cramér’s V is a more reliable signal than MI (which can be noisy for sparse distributions). For binary targets with nonlinear feature relationships, $\tilde{I}$ and π outperform ρ . The weighted combination systematically reduces false negatives compared to any single check applied in isolation.

7.4 Configurability and Reproducibility

By exposing all weights, thresholds, and penalty values through `config.yaml`, the framework enables full reproducibility: a given configuration file deterministically produces the same readiness score for the same dataset. This also supports sensitivity analysis—a user can sweep leakage weights or penalty values to understand how the score changes under different risk philosophies—without modifying any source code.

7.5 Limitations

The framework currently handles only classification tasks; regression requires a modified performance-inflation metric (e.g. R^2 delta). The per-feature CV in the unified risk score adds $O(p)$ overhead, noticeable on wide datasets ($p > 500$). The semantic module introduces a cloud API dependency, latency (5–15 seconds for 30 features), and a small cost ($\approx$ \$0.001 per call at GPT-4o-mini pricing). The quality of semantic analysis depends on the user-provided dataset description: brief descriptions reduce the model’s ability to distinguish temporal leakage from legitimate features.

7.6 Future Work

Four directions for future extension are identified. First, support for regression and time-series tasks requires new performance-inflation metrics and temporally-aware sufficiency checks. Second, causal discovery methods (PC algorithm, FCI) could detect indirect leakage paths that information-theoretic measures miss when the causal graph is non-trivial. Third, a larger semantic leakage benchmark derived from real data quality incident reports would enable more statistically robust evaluation across diverse domain vocabularies and naming conventions. Fourth, a data-driven calibration of the readiness score weights—learning from labelled examples of production failures—would make the score more adaptive to domain-specific risk profiles.

Chapter 8

Conclusions

This thesis presented **ml-framework**, a comprehensive open-source toolkit for automated dataset quality assessment and data leakage detection in machine learning workflows. The main contributions are:

1. A 22-check automated pipeline that covers quality, leakage, feature analysis, statistical sufficiency, drift detection, and performance impact in a single configurable system with a Python SDK, CLI, and Streamlit web interface.
2. A novel *unified leakage risk score* that combines Pearson/Cramér correlation, normalised mutual information, and per-feature performance inflation into an interpretable score $\mathcal{L}(f) \in [0, 1]$, addressing the limitation of single-signal heuristics.
3. An optional GPT-4o-mini semantic leakage module that detects implicit leakage from feature names, achieving $F1 = 0.963$ on a manually labelled 30-feature benchmark.
4. Empirical evidence that leakage causes measurable and recoverable performance degradation: removing the leaky feature from `leaky_dataset` reduces accuracy by $\Delta = -0.048$, confirming the model was exploiting leaked information.
5. A quantitative benchmark showing that `ml-framework` covers 29/29 ML-relevant checks and detects 4/4 leakage scenarios, while the best competing tool covers 11/29 checks and detects at most 1/4 leakage scenarios.
6. Fully configurable parameters (weights, penalties, thresholds) exposed through YAML to support reproducibility and sensitivity analysis.

The framework has been evaluated on eleven datasets—six synthetic and five real-world—demonstrating calibrated readiness scores across a range of defect profiles: from a clean control (95/A) to a leaky dataset (72/B) to a dataset with severe quality issues (65/C). Real-world case studies on Titanic, Adult Census, and German Credit confirm that the framework correctly identifies domain-specific problems, including the well-known boat leakage in Titanic and irrelevant features in German Credit.

All code, datasets, benchmarks, and the Streamlit demo are publicly available at <https://github.com/jaimecano12/ml-framework>.

Bibliography

- [1] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.
- [2] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [3] E. Breck, M. Polyzotis, S. Roy, S. Whang, and M. Zinkevich, “Data validation for machine learning,” in *Proc. MLSys*, 2019.
- [4] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [5] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. KDD*, 2016.
- [6] A. Kraskov, H. Stögbauer, and P. Grassberger, “Estimating mutual information,” *Physical Review E*, vol. 69, no. 6, 066138, 2004.
- [7] B. C. Ross, “Mutual information between discrete and continuous data sets,” *PLOS ONE*, vol. 9, no. 2, e87357, 2014.
- [8] S. Rabanser, S. Günnemann, and Z. Lipton, “Failing loudly: An empirical study of methods for detecting dataset shift,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [9] N. Siddiqi, *Credit Risk Scorecards*. Hoboken, NJ: Wiley, 2006.
- [10] W. McKinney, “Data structures for statistical computing in Python,” in *Proc. Python in Science Conference*, 2010.
- [11] OpenAI, “GPT-4 technical report,” arXiv:2303.08774, 2023.
- [12] D. Dua and C. Graff, *UCI Machine Learning Repository*. Irvine, CA: University of California, School of Information and Computer Science, 2017.
- [13] D. Zha et al., “Data-centric artificial intelligence: A survey,” arXiv:2303.10158, 2023.
- [14] A. Narayan, R. Chami, L. Orr, S. Arora, P. Bailis, and M. Zaharia, “Can Foundation Models Wrangle Your Messy Data?,” in *Proc. VLDB*, 2022.
- [15] X. Sui, P. Wu, J. Liu, M. Zhang, and A. Kumar, “Auto-Pipeline: Synthesizing Complex Data Science Pipelines by Prompt,” arXiv:2310.07298, 2023.
- [16] A. Ng, “A chat with Andrew on MLOps: From model-centric to data-centric AI,” *DeepLearning.AI talk*, 2021.

- [17] ydata-profiling contributors, “ydata-profiling: Profile your pandas DataFrame in one line,” <https://github.com/ydataai/ydata-profiling>, 2023.
- [18] Superconductive, “Great Expectations: Always know what to expect from your data,” https://github.com/great-expectations/great_expectations, 2023.
- [19] Deepchecks team, “Deepchecks: Testing and validating your ML models and data,” <https://github.com/deepchecks/deepchecks>, 2023.

Chapter A

Ethical, Economic, Social, and Environmental Aspects

A.1 Introduction

This appendix discusses the broader implications of an automated dataset quality assessment and leakage detection framework from ethical, economic, social, and environmental perspectives. Although ml-framework is primarily a software engineering and research contribution, its application in real-world machine learning pipelines raises considerations worth documenting.

A.2 Ethical Considerations

Bias detection and fairness. The class imbalance check and the feature relevance module can help practitioners identify datasets where minority groups are underrepresented. Early detection of imbalance—before model training—reduces the risk of deploying biased predictive models in high-stakes domains such as credit scoring, healthcare, and criminal justice. Practitioners are encouraged to treat the framework’s readiness score as one input among several in a broader data ethics review.

Transparency and reproducibility. By making all analysis parameters configurable through a public YAML file and producing interpretable check results with detailed messages, the framework supports transparent and reproducible ML workflows. This aligns with emerging regulatory requirements such as the EU AI Act, which mandates documentation and audit trails for high-risk AI systems.

LLM-based analysis. The semantic leakage module uses a large language model (GPT-4o-mini) to analyse feature names. While this module improves detection coverage, it introduces a dependency on a third-party API and the non-determinism inherent to LLM outputs. Practitioners should validate LLM findings with domain experts before acting on them, particularly in regulated industries.

A.3 Economic Considerations

The framework is open-source (MIT licence) and available at no cost. The main economic benefit is cost reduction: undetected data leakage can lead to model failures in production that are expensive to diagnose and remediate. The semantic leakage module incurs a small API cost (approximately \$0.001 per dataset call at current GPT-4o-mini pricing) when Azure OpenAI credentials are configured; the module is disabled by default.

A.4 Social Considerations

Automated dataset validation can democratise access to data quality tools previously available only in large technology companies with dedicated data engineering teams. By providing a free, easy-to-use SDK and an interactive Streamlit dashboard, ml-framework lowers the barrier to high-quality ML practice for academic researchers, small organisations, and individual practitioners.

A.5 Environmental Considerations

The framework’s computational footprint is modest. The 22-check pipeline completes in under 10 seconds on typical datasets (Table 6.8). The LLM semantic module, when enabled, makes a single API call to a cloud-hosted model; the environmental cost of this call is negligible relative to the cost of training a production ML model. The framework does not require GPU computation for any of its checks.

Chapter B

Economic Budget

B.1 Development Cost Estimation

Table B.1 provides an estimated development budget for the ml-framework project. The estimates are based on the hours of engineering work required for each project phase, evaluated at a junior-to-mid-level software engineering rate of €35/hour.

Table B.1. Estimated development budget.

Phase / Activity	Hours	Rate (€/h)	Cost (€)	Description
Scaffold & utilities (1–2)	20	35	700	Project structure, config, dataclasses
Quality checks (3)	15	35	525	6 quality checks + 44 tests
Leakage detection (4)	20	35	700	4 classical checks + 41 tests
Impact analysis (5)	15	35	525	CV baseline comparison
Report generation (6)	20	35	700	Jinja2 HTML + Matplotlib
Experiments & scripts (7)	10	35	350	Dataset generation scripts
Recommendations (8)	12	35	420	20 rule handlers
Feature analysis (9)	15	35	525	Correlation, MI, distribution
Readiness score (10)	10	35	350	Composite score + grades
Sufficiency (11)	12	35	420	4 statistical checks
Streamlit app (12)	20	35	700	7-tab web dashboard
Python SDK (13)	10	35	350	DatasetChecker class
Drift detection (14)	12	35	420	KS test + PSI
Plugin system (15)	8	35	280	@register_check API
Unified risk score + LLM (16)	30	35	1,050	Eq. 1 + semantic module
Benchmark + case studies (17)	20	35	700	Evaluation suite
Demo notebook	8	35	280	14-cell executed notebook
Research paper (LaTeX)	25	35	875	13-page conference paper
Thesis document (LaTeX)	30	35	1,050	Full TFM write-up
Total engineering	312		10,920	
Cloud API costs (Azure OpenAI)	—	—	5	Semantic leakage testing (~50 calls)
Software licences	—	—	0	All tools open-source
Hardware	—	—	0	Standard laptop, no GPU required
Total project cost			10,925	